

Informática Avanzada

Conceptos de Testing y Debugging

Errores en Software

- Tipos de Errores
 - Sintácticos
 - Los captura el compilador
 - Lógica
 - Producen el mal funcionamiento del sistema
 - Pueden ser más o menos directos
 - Diseño
 - Son en general semánticos
 - No son necesariamente de código
- La cantidad de errores que exista, nos da una idea de la calidad de nuestro sistema



Manejo de Errores

- Siempre existirán, pero pueden **mitigarse** (e.g. a través de Ingeniería del Software)
 - Encapsulación
 - Modularización
 - Documentación
- Los errores pueden detectarse a través de técnicas de testeo
 - Tests Manuales (smoke tests, tests funcionales)
 - Tests Automáticos
 - Unit Tests
 - Integration Test
- Una vez detectados un error, puede determinarse la causa a través técnicas de depuración
 - Seguimiento manual del código
 - Instrucciones de Impresión
 - Uso de Debugger



Modularización e Interfaces

- Las aplicaciones se componen con diferentes módulos
 - Diferentes equipos de personas puedan trabajar en ellos.
- La interfaz entre los módulos debe estar claramente especificada.
 - Permite el desarrollo concurrente en forma independiente.
 - Facilita la integración entre módulos
- Cada módulo no necesita conocer los detalles de la implementación del otro.



Niveles de Testing

- Las pruebas pueden realizarse a diferentes niveles
 - Unit testing
 - Component Testing
 - Integration Testing
 - System Testing
 - E2E testing
- Nuestro scope se va a limitar a los testeos de unidad
 - Métodos
 - Clases
 - Paquetes
- Debe realizarse conjuntamente con las tareas de desarrollo
 - Reduce costos y tiempo



Cómo desarrollar pruebas

- Qué testear?
- Probamos la **interfaz**
 - Entender qué debería hacer el método
 - Casos positivos y negativos
- Probar límites y condiciones de contorno
 - Rangos válidos e inválidos de valores
- Las buenas pruebas son un proceso creativo



Tests Automáticos

- Las pruebas exhaustivas consumen tiempo y son repetitivas.
 - Diseño de los tests
 - Ejecución de los tests
 - La regresión de los tests luego de cambios involucran la re-ejecución de los tests
- El uso de un esquema de tests puede disminuir algo de la complejidad.
 - Escribir clases para realizar la prueba.
 - La creatividad se enfoca en crearlas.
- Evitar el análisis humano de los resultados
- Uso de JUnit Framework

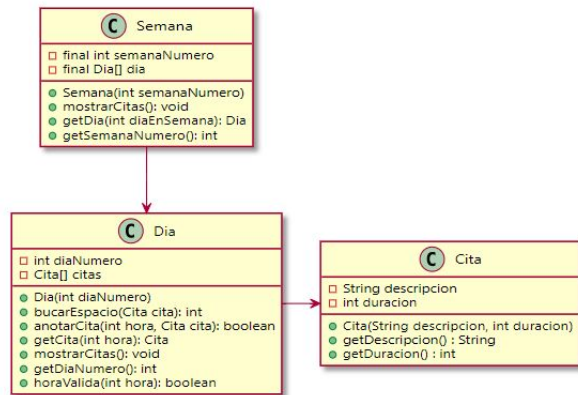
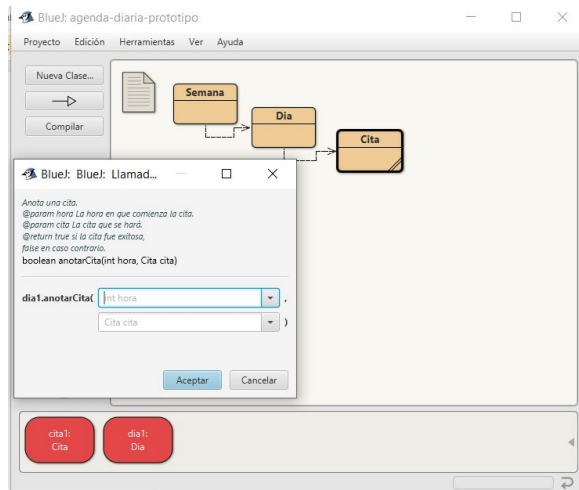


Técnicas de depuración

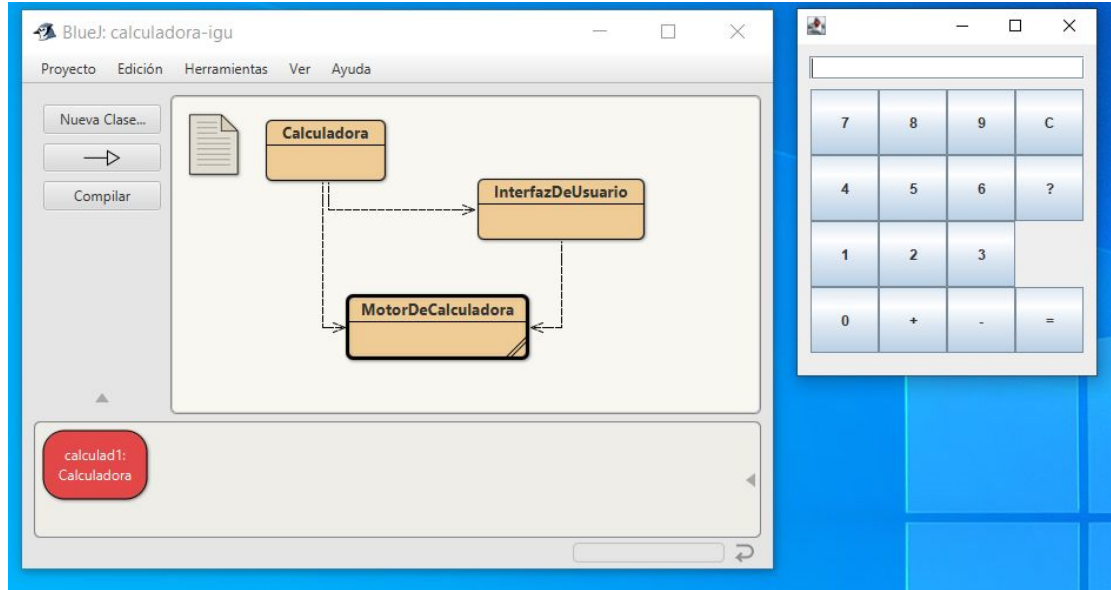
- Seguimiento manual
 - Ejecutar el código a mano
- Tabulación del estado del sistema
 - Mantener registro del estado de cada objeto
- Seguimiento verbal (Peer Programming)
 - Explicar a otra persona el flujo o comportamiento esperado del código
- Impresión de logs para seguimiento
 - Puede generar grandes volúmenes de datos
 - Existen
- Uso de debuggers



Proyecto “Agenda Diaria”



Proyecto Calculadora



: Lote

int numero	1
String desc	Peugeto
Oferta ofertaMax	null

visorDeR1: VisorDeReloj

private VisorDeNumeros horas	
private VisorDeNumeros minutos	
private String cadVisor	

:ItemDeCorreo

de	Ana
para	Juan
mensaje	Hola

